

How the booster lander works

Bringing a spent first stage back to the KSC launch pad, Super Heavy style, from a kRPC script

The booster separates at about 20 km, still moving downrange at several hundred metres per second. The autopilot flips it, burns back toward the pad, falls through the atmosphere using the body as a weak wing, and lights the engines once more for a suicide burn that puts the legs on a pad about 20 m across. Over five different separation states, with no calibration constant, it lands a median of **4 m** from the target and **7 m** at worst.

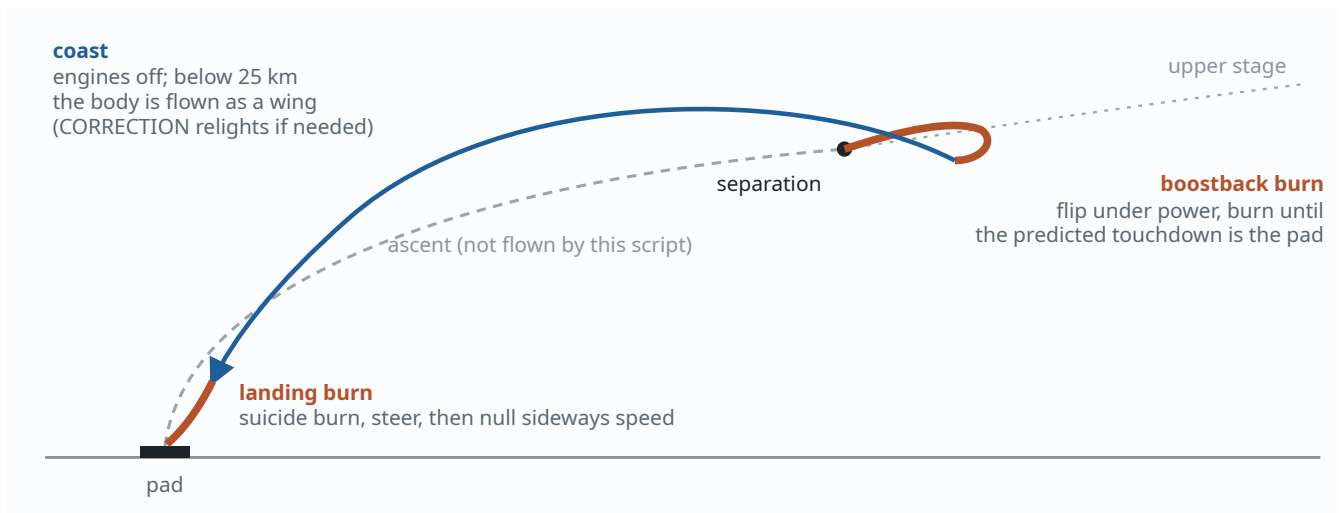


Figure 1. The flight profile, not to scale. Orange is engine-on; blue is unpowered. Every part of it is re-decided each tick from a fresh prediction of where the booster will touch down.

The one idea: re-predict the landing every tick

There is no stored plan. About twenty times a second the autopilot reads the booster's position and velocity, then **integrates the rest of the flight to the ground** in local Python: gravity, drag, body lift, the planet's rotation, and the landing burn itself. Whatever it decides that tick, it decides from that fresh prediction: which way to point, how much throttle, when boostback is done, when to light the landing burn, which way to steer. A disturbance that sends the booster off course shows up in the next prediction, so it gets corrected without anything having to detect it as a disturbance.

The code is organised around that idea. `autoland.py` is the phase machine and the only code that talks to the game. `trajectory.py` is the propagator. `guidance.py` holds the control laws as pure functions that can be tested without KSP. `environment.py` keeps the world data the propagator needs: air density, speed of sound, a drag curve and a lift slope. It caches all of it so the integration loop makes no remote calls at all.

What the predictor has to get right

- **It flies the landing burn too.** The prediction coasts to where the suicide burn will start, then keeps integrating with the engines on, using the same throttle law the vehicle will fly (`trajectory.landing_command` is shared by both). Braking sheds horizontal speed, so the real touchdown is short of the ballistic impact point, by kilometres for a fast booster. A boostback that aimed at the ballistic point would overshoot every time.

- **RK4, not Euler.** Drag goes as v^2 through a density that changes by a factor of e every 5 km. A first-order step over a hundred-second descent *biases* the answer instead of just adding noise. That bias was 60–100 m of overshoot, which a hand-fitted aim correction had been quietly cancelling.
- **Drag is a curve against Mach, not a number.** One flight measured $C_d \cdot A$ at 15 m² subsonic, 50 m² near Mach 1.2 and 20 m² at Mach 2.5. The curve is probed with kRPC's `simulate_aerodynamic_force_at`, which takes the velocity to evaluate as an argument, so the whole curve can be measured from one position. It is re-swept once the booster is falling nose-first, because a drag area measured broadside during boostback is wrong for the descent.
- **The planet's rotation is measured, not assumed.** kRPC frames are left-handed, so the sign of a cross product is a trap. The rotation vector is found by comparing the vessel's velocity in the rotating and inertial frames and picking the one that matches.
- **Heights are measured from the feet.** The trigger height uses the terrain under the legs, clamped against the pad's own radius. One poisoned sensor reading had ended four flights.

The flight, phase by phase

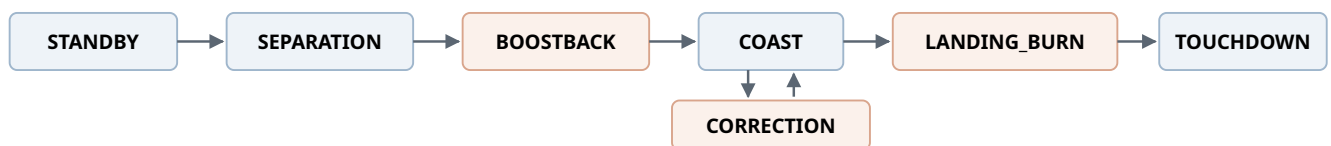


Figure 2. The phase machine. Orange phases burn the engines. CORRECTION is a short relight that COAST can enter and return from.

STANDBY

waits for START

Idle until the in-game panel's START button. Then: RCS on, action group 2 (grid fins and airbrakes), and the autopilot engaged on the current attitude.

SEPARATION

a few seconds

The question is *when it is safe to flip*. The first version waited three seconds, which is wrong in both directions: too short and the booster turns into its own upper stage; too long and every second is more downrange distance the boostback has to undo. `proximity.py` asks directly. It wraps each nearby craft in a sphere and marches the booster's motion forward against each one, both coasting and at full thrust along the boostback aim. The phase ends on the first tick where nothing is in the way.

One subtlety drives the whole model. A moment before separation the two stages were one vehicle, so their spheres overlap at $t = 0$ by construction. The keep-out distance is therefore capped at the gap they are already sitting at, which is demonstrably survivable. Only getting *closer* counts as a conflict. A 5 s backstop goes anyway: a booster that never flips is certainly lost.

BOOSTBACK

flip and burn

Aim opposite the predicted horizontal miss and go to **full throttle on the first tick, before the turn is finished**. The gimbal is the strongest attitude actuator the booster has, and it exists only while the engines are running. An early version flipped a 30 t booster at 19 km and 584 m/s with the engines off and wallowed through 150° on wheels and RCS. Big turns are routed *over the top*: kRPC turns along the shortest arc, and for a near-180° flip that is a coin toss which can swing the engines down into the airstream.

Throttle tapers off as the remaining Δv falls, and the burn ends when the predicted touchdown, landing burn included, is within 190 m of the pad. Roll is commanded as well as pointing, so the grid fins sit in the plane they steer in.

COAST

engines off

Fall back toward the pad, watching for the moment the landing burn has to start. Above 25 km it holds retrograde. Below 25 km it **flies the body as a wing** (next section), which is the single largest accuracy gain in the project.

CORRECTION

relight, if needed

If the predicted miss grows past 60 m while there is still height to use, a throttled-down relight closes it to under 15 m and returns to COAST. A falling booster can't hold boostback's horizontal attitude in thick air, so it steers within 50° of retrograde. Before each burn it *propagates twice* to measure how many metres of miss one m/s actually closes, and it never fires a burn that would make the miss worse. If the booster is stuck off-aim and not turning, it lights at 25% throttle so the gimbal can turn it.

LANDING_BURN

suicide burn

Gear down on the first tick. Throttle follows a constant-deceleration profile (feed-forward plus a proportional trim) that blends into a constant-rate final descent, so the legs arrive at 2 m/s. The burn also steers, with ZEM/ZEV terminal guidance aimed at the propagator's predicted touchdown. Below 250 m it **stops aiming and only nulls sideways velocity** (see “Aim, then stop”).

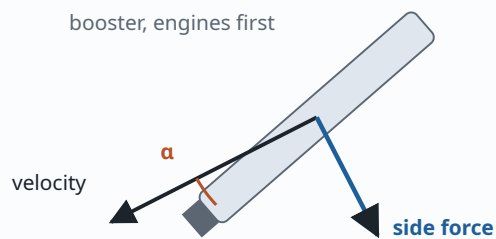
TOUCHDOWN

hand back

Throttle zero, log the distance from the pad, exit. Every exit path (TERMINATE, Ctrl-C, a crash) runs the same shutdown: throttle zero, autopilot off, SAS on, panel removed, connection closed.

Flying the coast as a wing

Every other actuator is the engine, and engine burns can only *shorten* the flight: CORRECTION steers near retrograde and the landing burn thrusts against the velocity. An undershoot was therefore a miss nothing could fix. One separation state kept landing 130–160 m short, and the prediction showed it from 34 km up with fuel still aboard. Meanwhile the booster spends a minute falling through air thick enough to decelerate it at more than 1 g. Held a few degrees off retrograde, that air is an actuator that pushes in **both** directions, costs no propellant, and acts for the whole time the miss is already known.



Solve, then verify

1. Measure the side-force slope with the same probe that measures drag (no lift found = hold retrograde).
2. Propagate small test angles in-plane and out-of-plane; invert the 2x2 to null downrange and cross-track.
3. Propagate the answer. Keep it only if it lands nearer than doing nothing (else halve it once, then give up).

Figure 3. Tilting the booster a few degrees off its velocity turns the airflow into a sideways force. The force goes roughly as $\alpha \cdot |\alpha|$ (a cylinder in crossflow, not a wing), so both the propagator and the solver are quadratic in the angle.

Nothing assumes the booster has lift. The slope is *measured* each flight, and a vehicle with none is simply not steered. What this booster has turned out to be body lift rather than grid-fin lift (fins stowed and deployed give identical side force; what the fins change is drag), with a lift-to-drag ratio of only about 0.04. That is still ample: 0.4 m/s^2 over the last 40 s is about 300 m of authority against a typical 130 m error.

TWO GUARDS THE RESULT DEPENDS ON

Wait for the prediction to settle. The wing starts at 25 km, not at the boostback exit. Right after boostback the prediction still walks by up to ~180 m while the drag curve is re-measured nose-first. A wing that steered against that walk removed an error that was going to correct itself, and steering immediately cost about 100 m.

Check the answer before flying it. Near the ground the sensitivity collapses, so an already-closed miss divided by almost nothing comes back as the maximum angle. One flight held the miss at 0–5 m from 32 km and then threw it to 19 m in a single step. Re-propagating the command and refusing one that lands further away is free, because that prediction is computed anyway.

SEPARATION STATE	NO WING (M)	WITH WING (M)
quicksave		25
qs_hot		68
qs_cold		14
qs_north		15
qs_steep		151

Table 1. Distance from the pad, two in-game flights per cell, aim bias zeroed so this is guidance error rather than calibration.

Aim, then stop

The landing burn steers with ZEM/ZEV (zero-effort-miss / zero-effort-velocity) guidance. Its gains, 6 and 4, come from the energy-optimal solution rather than from tuning, and the tilt is capped so that steering never takes thrust away from braking. When it was first flown it **toppled the booster**. Neither the gains nor the cap was the problem. The steering simply switched off at 15 m, and the sideways speed it had built while aiming was still on the vehicle when the legs arrived: 3.4 m/s at 16 m up, against 0.1 for a pure retrograde burn.

So below 250 m the position term is dropped and the same law runs on its velocity term alone, down to 4 m. Aiming and stopping are different jobs, and the last stretch belongs to stopping. A sim log shows the mechanism in its horizontal-speed column: 3.5, 5.0, 9.6 (aiming), 5.8, 0.7, 0.1, 0.0. The simulator

scores the guard as costing 17 m, because its vehicle points instantly and cannot fall over. The configuration it prefers is the one that broke the booster in game.

Smaller rules that each cost a flight

- **Burn for the gimbal.** If CORRECTION is far off its aim *and not turning toward it* (under 3 °/s), it lights at 25% throttle rather than waiting. One flight spent 36 s pointed 45–72° away, did nothing, and watched the miss grow from 341 to 966 m. The rate test matters: burning whenever the booster is merely off-aim wastes propellant on a turn RCS was already making.
- **RCS is a relay, not a phase setting.** Thrusters open when the pointing error is large and close once it has *stayed* small for a moment. A single threshold would chatter at the frequency of the oscillation they are damping.
- **Roll is held, not left free.** The roof is rolled into the trajectory plane so the grid fins steer the axis that matters. The roll reference is seeded from the zenith once and then carried forward; re-deciding it every tick would flip it 180° as the nose passes vertical and command a barrel roll.
- **The landing burn does not divert.** An early version used the landing burn to remove the remaining miss and landed the booster on its side. Boostback and the coast remove the miss; the landing burn stops the vehicle.

How it is measured

Each flight writes a numbered log (`logs/LOG<n>`). Telemetry is paced in *game* time, so a paused game logs nothing and time warp still gives one line per interval. Every log records which settings differ from the defaults *and* a fingerprint of the defaults themselves, so a flight can always be filed under the right configuration. Tunables live only in `config.py`; `--set FIELD=VALUE` overrides any of them for one flight.

Five reference saves (`quicksave`, `qs_hot`, `qs_cold`, `qs_steep`, `qs_north`) give five different separation states, and `tools/savegen.py` writes more. An offline simulator (`tests/fakeksp`) runs the full closed loop with no game, but it models a point mass with near-instant pointing and no aerodynamic torque. Six changes that improved a sim sweep made the real flight worse, so every sim result is treated as a hypothesis and confirmed by flying it in KSP (`tools/quickfly.py`, or the multi-instance farm in `testInstances/`).

Written 2026-10-05 from `docs/boosterland/design.md`, `failures.md` and the code at commit `8ae26ff`. Configuration values quoted are the committed defaults in `boosterland/config.py`.